

Statistical Variance Reduction, Non-Parametric Bootstrap Sampling, and Empirical Functional Aggregating

A Rigorous Mathematical, Information-Theoretic, and Physical Foundation
of Pure Bagging

Contents

1	The Concept of Pure Bootstrap Aggregating (Bagging)	2
2	Rigorous Proof of Variance Reduction and Bias Preservation	2
2.1	Bias-Variance Decomposition of MSE	2
2.2	Proof of Bias Preservation	2
2.3	Proof of Variance Reduction	3
3	The Non-Parametric Bootstrap and Out-of-Bag (OOB) Theory	3
3.1	Mathematical Proof of the Out-of-Bag (OOB) Multiplier	4
3.2	OOB Cross-Validation Mathematics	4
4	Physical Analogies: Statistical Mechanics & Monte Carlo Phase Space	5
4.1	Gibbs Ensemble Averaging in Phase Space	5
4.2	Monte Carlo Integration over Hypothesis Space	5
5	Aggregating Schemes: Regression vs. Classification	5
5.1	Continuous Regression: Arithmetic Expectation	6
5.2	Classification: Hard Voting vs. Soft Probability Averaging	6
6	Production-Grade Vectorized NumPy Implementation	6

1 The Concept of Pure Bootstrap Aggregating (Bagging)

In supervised machine learning, models with high capacity—such as deep, unpruned decision trees—exhibit low structural bias but exceptionally high variance. Small fluctuations in the empirical training set $\mathcal{D}$ lead to drastic shifts in the constructed hypothesis $f(\mathbf{x})$.

****Bootstrap Aggregating (Bagging)****, introduced by Leo Breiman in 1996, is a pure, domain-agnostic meta-algorithm designed strictly to reduce the variance of unstable base estimators without altering their expectation (bias).

Unlike broader ensemble architectures that modify feature spaces or sequentially adjust target residuals, pure Bagging operates exclusively through ****parallel empirical resampling**** and ****functional averaging****.

Definition 1.1 (Pure Bagging Estimator). Let $\mathcal{D} = \{(\mathbf{x}^{(1)}, y^{(1)}), (\mathbf{x}^{(2)}, y^{(2)}), \dots, (\mathbf{x}^{(N)}, y^{(N)})\}$ be an empirical training dataset of N i.i.d. observations sampled from an underlying probability distribution $p_{\text{data}}(\mathbf{x}, y)$. Let $g(\mathbf{x}; \mathcal{D})$ denote a base learning algorithm trained on $\mathcal{D}$.

Given M independent bootstrap datasets $\mathcal{D}^{*(1)}, \mathcal{D}^{*(2)}, \dots, \mathcal{D}^{*(M)}$ constructed by sampling N points from $\mathcal{D}$ uniformly with replacement, the pure Bagging functional estimator $\hat{f}_{\text{bag}}(\mathbf{x})$ is:

$$\text{Regression: } \hat{f}_{\text{bag}}(\mathbf{x}) = \frac{1}{M} \sum_{m=1}^M g(\mathbf{x}; \mathcal{D}^{*(m)}) \quad (1)$$

$$\text{Classification (Hard Voting): } \hat{f}_{\text{bag}}(\mathbf{x}) = \arg \max_{c \in \{1, \dots, C\}} \sum_{m=1}^M \mathbb{I}(g(\mathbf{x}; \mathcal{D}^{*(m)}) = c) \quad (2)$$

$$\text{Classification (Soft Voting): } \hat{f}_{\text{bag}}(\mathbf{x}) = \arg \max_{c \in \{1, \dots, C\}} \frac{1}{M} \sum_{m=1}^M P_g(Y = c \mid \mathbf{x}; \mathcal{D}^{*(m)}) \quad (3)$$

2 Rigorous Proof of Variance Reduction and Bias Preservation

To understand why Bagging stabilizes predictions, we analyze its effect on the ****Bias-Variance Decomposition**** under Mean Squared Error (MSE).

2.1 Bias-Variance Decomposition of MSE

Let $y = f(\mathbf{x}) + \epsilon$ be the true target function, where noise ϵ satisfies $\mathbb{E}[\epsilon] = 0$ and $\text{Var}(\epsilon) = \sigma_\epsilon^2$. The expected generalization error of a base model $\hat{g}(\mathbf{x}) = g(\mathbf{x}; \mathcal{D})$ at point $\mathbf{x}$ decomposes into:

$$\mathbb{E}_{\mathcal{D}} [(y - \hat{g}(\mathbf{x}))^2] = \underbrace{(f(\mathbf{x}) - \mathbb{E}_{\mathcal{D}}[\hat{g}(\mathbf{x})])^2}_{\text{Bias}^2} + \underbrace{\mathbb{E}_{\mathcal{D}} [(\hat{g}(\mathbf{x}) - \mathbb{E}_{\mathcal{D}}[\hat{g}(\mathbf{x})])^2]}_{\text{Variance}} + \underbrace{\sigma_\epsilon^2}_{\text{Irreducible Error}} \quad (4)$$

2.2 Proof of Bias Preservation

Theorem 2.1 (Bias Invariance of Bagging). Under ideal bootstrap conditions where each bootstrap sample $\mathcal{D}^*$ follows the empirical distribution $\hat{P}_N$, the expected prediction of the Bagging estimator equals the expected prediction of a single base model trained on $\mathcal{D}$.

Proof. Taking the expectation of the Bagging estimator with respect to the bootstrap sampling process $\mathcal{D}^*$:

$$\mathbb{E}_{\mathcal{D}^*} [\hat{f}_{\text{bag}}(\mathbf{x})] = \mathbb{E}_{\mathcal{D}^*} \left[\frac{1}{M} \sum_{m=1}^M g(\mathbf{x}; \mathcal{D}^{*(m)}) \right] = \frac{1}{M} \sum_{m=1}^M \mathbb{E}_{\mathcal{D}^*} [g(\mathbf{x}; \mathcal{D}^{*(m)})] \quad (5)$$

Since each bootstrap set $\mathcal{D}^{*(m)}$ is generated by an identical resampling procedure from $\mathcal{D}$:

$$\mathbb{E}_{\mathcal{D}^*} [g(\mathbf{x}; \mathcal{D}^{*(m)})] = \mathbb{E}_{\mathcal{D}^*} [g(\mathbf{x}; \mathcal{D}^*)] \quad (6)$$

Thus:

$$\mathbb{E}_{\mathcal{D}^*} [\hat{f}_{\text{bag}}(\mathbf{x})] = \mathbb{E}_{\mathcal{D}^*} [g(\mathbf{x}; \mathcal{D}^*)] \quad (7)$$

Subtracting $f(\mathbf{x})$ from both sides shows that the structural bias remains invariant:

$$\text{Bias}(\hat{f}_{\text{bag}}(\mathbf{x})) = \text{Bias}(g(\mathbf{x}; \mathcal{D}^*)) \quad (8)$$

□

2.3 Proof of Variance Reduction

Let $\sigma^2(\mathbf{x}) = \text{Var}_{\mathcal{D}^*}(g(\mathbf{x}; \mathcal{D}^*))$ be the variance of a single base estimator. Because all bootstrap sets $\mathcal{D}^{*(m)}$ are sampled from the same dataset $\mathcal{D}$, the predictions of different base models are not completely independent; they share pairwise correlation $\rho(\mathbf{x}) = \text{Corr}(g(\mathbf{x}; \mathcal{D}^{*(j)}), g(\mathbf{x}; \mathcal{D}^{*(k)}))$ for $j \neq k$.

Theorem 2.2 (Variance Reduction Formula for Pure Bagging). The variance of the pure Bagging estimator is given by:

$$\text{Var}(\hat{f}_{\text{bag}}(\mathbf{x})) = \rho(\mathbf{x})\sigma^2(\mathbf{x}) + \frac{1 - \rho(\mathbf{x})}{M}\sigma^2(\mathbf{x}) \quad (9)$$

Proof.

$$\begin{aligned} \text{Var}(\hat{f}_{\text{bag}}(\mathbf{x})) &= \text{Var}\left(\frac{1}{M} \sum_{m=1}^M g(\mathbf{x}; \mathcal{D}^{*(m)})\right) \\ &= \frac{1}{M^2} \sum_{j=1}^M \sum_{k=1}^M \text{Cov}(g(\mathbf{x}; \mathcal{D}^{*(j)}), g(\mathbf{x}; \mathcal{D}^{*(k)})) \\ &= \frac{1}{M^2} \left[\sum_{m=1}^M \text{Var}(g(\mathbf{x}; \mathcal{D}^{*(m)})) + \sum_{j=1}^M \sum_{k \neq j}^M \text{Cov}(g(\mathbf{x}; \mathcal{D}^{*(j)}), g(\mathbf{x}; \mathcal{D}^{*(k)})) \right] \\ &= \frac{1}{M^2} [M\sigma^2(\mathbf{x}) + M(M-1)\rho(\mathbf{x})\sigma^2(\mathbf{x})] \\ &= \frac{\sigma^2(\mathbf{x})}{M} + \frac{M-1}{M}\rho(\mathbf{x})\sigma^2(\mathbf{x}) \\ &= \rho(\mathbf{x})\sigma^2(\mathbf{x}) + \frac{1 - \rho(\mathbf{x})}{M}\sigma^2(\mathbf{x}) \end{aligned} \quad (10)$$

□

3 The Non-Parametric Bootstrap and Out-of-Bag (OOB) Theory

Because true data sampling from p_{data} is unavailable, Bagging utilizes Bradley Efron's ****Non-Parametric Plug-in Principle**** (1979), substituting the unknown continuous distribution p_{data} with the empirical distribution $\hat{P}_N$:

$$\hat{P}_N(\mathbf{X} = \mathbf{x}, Y = y) = \frac{1}{N} \sum_{i=1}^N \delta(\mathbf{x} - \mathbf{x}^{(i)}, y - y^{(i)}) \quad (11)$$

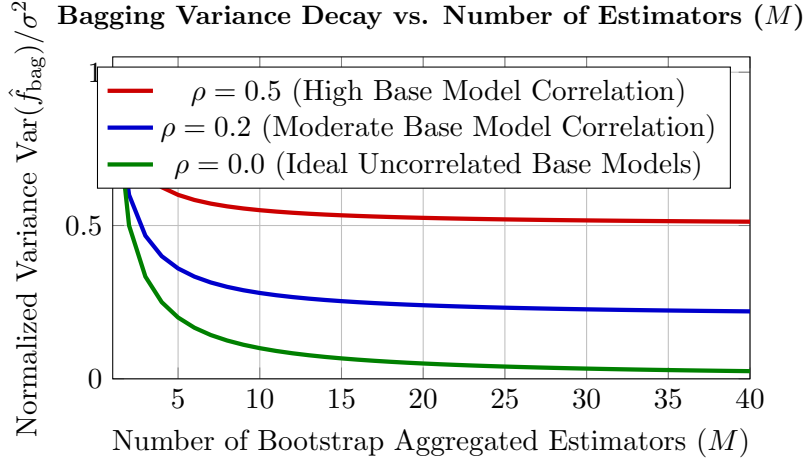


Figure 1: Variance decay curves under Bagging for varying levels of inter-model correlation ρ . As $M \rightarrow \infty$, variance approaches the lower asymptote $\rho\sigma^2$.

3.1 Mathematical Proof of the Out-of-Bag (OOB) Multiplier

Each bootstrap dataset $\mathcal{D}^{*(m)}$ is constructed by drawing N uniform random samples from $\mathcal{D}$ with replacement.

Theorem 3.1 (Out-of-Bag Inclusion Limit). The probability that any specific training observation $(\mathbf{x}^{(i)}, y^{(i)})$ is omitted from a bootstrap dataset $\mathcal{D}^{*(m)}$ of size N converges asymptotically to $1/e \approx 0.367879$.

Proof. For a single draw, the probability of selecting observation i is $\frac{1}{N}$. The probability of *not* selecting observation i is $1 - \frac{1}{N}$.

Since draws are independent and performed with replacement, the probability P_{OOB} that observation i is omitted across all N draws is:

$$P_{\text{OOB}}(N) = \left(1 - \frac{1}{N}\right)^N \quad (12)$$

Taking the limit as dataset size $N \rightarrow \infty$:

$$\lim_{N \rightarrow \infty} P_{\text{OOB}}(N) = \lim_{N \rightarrow \infty} \left(1 - \frac{1}{N}\right)^N = \frac{1}{e} \approx 0.3678794411... \quad (13)$$

The fraction of unique training samples included in the bootstrap sample is:

$$P_{\text{in}} = 1 - \frac{1}{e} \approx 0.6321205588... \quad (63.2\%) \quad (14)$$

□

3.2 OOB Cross-Validation Mathematics

Because each bootstrap model $g(\mathbf{x}; \mathcal{D}^{*(m)})$ leaves out approximately 36.8% of training samples, these excluded instances act as an independent test set for that specific estimator.

Let $\mathcal{B}^{(i)} \subset \{1, 2, \dots, M\}$ be the index set of bootstrap models trained *without* sample $(\mathbf{x}^{(i)}, y^{(i)})$:

$$\mathcal{B}^{(i)} = \left\{ m \in \{1, \dots, M\} \mid (\mathbf{x}^{(i)}, y^{(i)}) \notin \mathcal{D}^{*(m)} \right\} \quad (15)$$

The **Out-of-Bag Predictor** $\hat{y}_{\text{OOB}}^{(i)}$ for training instance i is:

$$\hat{y}_{\text{OOB}}^{(i)} = \frac{1}{|\mathcal{B}^{(i)}|} \sum_{m \in \mathcal{B}^{(i)}} g(\mathbf{x}^{(i)}; \mathcal{D}^{*(m)}) \quad (16)$$

The ****OOB Generalization Error**** evaluates loss across all N out-of-bag evaluations:

$$\mathcal{R}_{\text{OOB}} = \frac{1}{N} \sum_{i=1}^N L\left(y^{(i)}, \hat{y}_{\text{OOB}}^{(i)}\right) \quad (17)$$

Theorem 3.2 (Unbiasedness of OOB Risk). The Out-of-Bag risk $\mathcal{R}_{\text{OOB}}$ is an asymptotically unbiased estimate of the true generalization error, providing performance metrics comparable to N -fold leave-one-out cross-validation without requiring additional computational passes.

4 Physical Analogies: Statistical Mechanics & Monte Carlo Phase Space

The mathematical formulation of Bagging aligns closely with fundamental principles in statistical mechanics and thermodynamics.

4.1 Gibbs Ensemble Averaging in Phase Space

In statistical thermodynamics, determining the macroscopic properties of a complex system by tracking a single trajectory over time is computationally intractable. Josiah Willard Gibbs resolved this by introducing ****Ensemble Averaging**** over macroscopic thermodynamic states.

Definition 4.1 (Gibbs Canonical Ensemble). Let $\mathbf{q} \in \Omega$ represent a microstate configuration in phase space. The macroscopic expectation of a physical observable $A(\mathbf{q})$ under thermal equilibrium at temperature T is:

$$\langle A \rangle = \int_{\Omega} A(\mathbf{q}) P(\mathbf{q}) d\mathbf{q} = \frac{1}{Z} \int_{\Omega} A(\mathbf{q}) e^{-E(\mathbf{q})/k_B T} d\mathbf{q} \quad (18)$$

where $Z = \int e^{-E(\mathbf{q})/k_B T} d\mathbf{q}$ is the canonical partition function.

In Bagging:

- A single training set $\mathcal{D}$ represents a single noisy trajectory or realization of data.
- Resampling datasets $\mathcal{D}^{*(m)}$ simulates drawing microstates $\mathbf{q}^{(m)}$ from a Gibbs distribution in configuration space.
- The aggregated predictor $\hat{f}_{\text{bag}}(\mathbf{x}) = \frac{1}{M} \sum g(\mathbf{x}; \mathcal{D}^{*(m)})$ calculates the ****Gibbs Ensemble Average**** $\langle A \rangle$, filtering out high-frequency thermal fluctuations associated with finite data sampling.

4.2 Monte Carlo Integration over Hypothesis Space

Bagging can be viewed as a ****Monte Carlo Integration**** technique over an empirical hypothesis space $\mathcal{H}$.

To compute the expected hypothesis $\bar{g}(\mathbf{x}) = \int_{\mathcal{H}} g(\mathbf{x}; \mathcal{D}) P(\mathcal{D}) d\mathcal{D}$, Bagging evaluates a finite-sample Monte Carlo approximation:

$$\bar{g}(\mathbf{x}) \approx \frac{1}{M} \sum_{m=1}^M g\left(\mathbf{x}; \mathcal{D}^{*(m)}\right) \quad (19)$$

Standard Monte Carlo error bounds state that the integration error decreases at a rate of $O(1/\sqrt{M})$, matching the variance decay rate of independent ensemble components.

5 Aggregating Schemes: Regression vs. Classification

The aggregation operator used in Bagging depends on whether the underlying task is continuous regression or discrete classification.

5.1 Continuous Regression: Arithmetic Expectation

For regression, Bagging evaluates the simple arithmetic mean of all M base predictions:

$$\hat{f}_{\text{bag}}(\mathbf{x}) = \frac{1}{M} \sum_{m=1}^M g(\mathbf{x}; \mathcal{D}^{*(m)}) \quad (20)$$

By Jensen's Inequality, if loss function $L(\cdot)$ is convex, the loss of the aggregated prediction is bounded above by the average loss of individual base predictions:

$$L\left(y, \frac{1}{M} \sum_{m=1}^M g_m(\mathbf{x})\right) \leq \frac{1}{M} \sum_{m=1}^M L(y, g_m(\mathbf{x})) \quad (21)$$

5.2 Classification: Hard Voting vs. Soft Probability Averaging

1. Hard Majority Voting

For discrete classes $c \in \{1, 2, \dots, C\}$:

$$\hat{f}_{\text{bag}}^{\text{hard}}(\mathbf{x}) = \arg \max_{c \in \{1, \dots, C\}} \sum_{m=1}^M \mathbb{I}(g(\mathbf{x}; \mathcal{D}^{*(m)}) = c) \quad (22)$$

2. Soft Probability Averaging

When base estimators output calibrated class probability vectors $\mathbf{p}_m(\mathbf{x}) = [p_{m1}, \dots, p_{mC}]^T$:

$$\hat{f}_{\text{bag}}^{\text{soft}}(\mathbf{x}) = \arg \max_{c \in \{1, \dots, C\}} \frac{1}{M} \sum_{m=1}^M P_g(Y = c \mid \mathbf{x}; \mathcal{D}^{*(m)}) \quad (23)$$

Soft probability averaging generally outperforms hard voting because it preserves confidence information from individual base classifiers rather than thresholding predictions prematurely.

6 Production-Grade Vectorized NumPy Implementation

The following Python class implements a pure, production-grade Bagging Engine compatible with any base estimator interface. It includes vectorized bootstrap execution, continuous OOB error monitoring, and support for both classification and regression.

```
1 import numpy as np
2
3 class UniversalBaggingEngine:
4     """
5     Production-grade Pure Bootstrap Aggregating (Bagging) Framework.
6     Implements non-parametric bootstrap resampling, functional aggregation,
7     and Out-of-Bag (OOB) error estimation for arbitrary base estimators.
8     """
9     def __init__(self, base_estimator_class, estimator_params: dict = None,
10                  n_estimators: int = 50, mode: str = 'classification'):
11         self.base_estimator_class = base_estimator_class
12         self.estimator_params = estimator_params if estimator_params is not
13         None else {}
14         self.n_estimators = n_estimators
15         self.mode = mode
16
17         self.estimators_ = []
18         self.oob_score_ = None
19         self.classes_ = None
```

```

19 def fit(self, X: np.ndarray, y: np.ndarray):
20     """Fits Bagging ensemble over N bootstrap iterations and evaluates
21     OOB score."""
22     X = np.asarray(X, dtype=np.float64)
23     y = np.asarray(y)
24     N = X.shape[0]
25
26     if self.mode == 'classification':
27         self.classes_ = np.unique(y)
28
29     self.estimators_ = []
30     oob_predictions = [[] for _ in range(N)]
31
32     for m in range(self.n_estimators):
33         # 1. Non-Parametric Bootstrap Draw (Sample N with replacement)
34         bootstrap_indices = np.random.choice(N, size=N, replace=True)
35
36         # Identify Out-of-Bag (OOB) samples
37         oob_mask = np.ones(N, dtype=bool)
38         oob_mask[bootstrap_indices] = False
39         oob_indices = np.where(oob_mask)[0]
40
41         X_boot = X[bootstrap_indices]
42         y_boot = y[bootstrap_indices]
43
44         # 2. Instantiate and fit base estimator on bootstrap sample
45         estimator = self.base_estimator_class(**self.estimator_params)
46         estimator.fit(X_boot, y_boot)
47         self.estimators_.append(estimator)
48
49         # 3. Accumulate predictions on OOB validation set
50         if len(oob_indices) > 0:
51             preds_oob = estimator.predict(X[oob_indices])
52             for idx, pred in zip(oob_indices, preds_oob):
53                 oob_predictions[idx].append(pred)
54
55         # 4. Evaluate Global Out-of-Bag (OOB) Score
56         oob_true, oob_pred = [], []
57         for idx in range(N):
58             if len(oob_predictions[idx]) > 0:
59                 oob_true.append(y[idx])
60                 if self.mode == 'classification':
61                     # Majority vote among OOB predictions
62                     counts = np.bincount(np.array(oob_predictions[idx],
dtype=int))
63                     oob_pred.append(np.argmax(counts))
64                 else:
65                     # Mean response among OOB predictions
66                     oob_pred.append(np.mean(oob_predictions[idx]))
67
68         if self.mode == 'classification':
69             self.oob_score_ = np.mean(np.array(oob_true) == np.array(
oob_pred))
70         else:
71             self.oob_score_ = np.mean((np.array(oob_true) - np.array(
oob_pred))**2)
72
73     return self
74

```

```

75     def predict_proba(self, X: np.ndarray) -> np.ndarray:
76         """Evaluates ensemble soft probabilities for classification."""
77         if self.mode != 'classification':
78             raise ValueError("predict_proba is available only for mode='
classification'.")
79
80         X = np.asarray(X, dtype=np.float64)
81         N_samples = X.shape[0]
82         K = len(self.classes_)
83
84         prob_sum = np.zeros((N_samples, K))
85
86         for estimator in self.estimators_:
87             if hasattr(estimator, "predict_proba"):
88                 prob_sum += estimator.predict_proba(X)
89             else:
90                 # One-hot fallback if base estimator lacks predict_proba
91                 preds = estimator.predict(X).astype(int)
92                 one_hot = np.zeros((N_samples, K))
93                 one_hot[np.arange(N_samples), preds] = 1.0
94                 prob_sum += one_hot
95
96         return prob_sum / self.n_estimators
97
98     def predict(self, X: np.ndarray) -> np.ndarray:
99         """Aggregates predictions across all base models."""
100         X = np.asarray(X, dtype=np.float64)
101
102         if self.mode == 'classification':
103             probs = self.predict_proba(X)
104             max_indices = np.argmax(probs, axis=1)
105             return self.classes_[max_indices]
106         else:
107             preds_matrix = np.array([est.predict(X) for est in self.
estimators_])
108             return np.mean(preds_matrix, axis=0)
109
110
111 # Mock Unstable Base Estimator for Execution Verification
112 class DummyUnstableTree:
113     def __init__(self, max_depth=8):
114         self.max_depth = max_depth
115         self.mean_val = 0.0
116
117     def fit(self, X, y):
118         self.mean_val = np.mean(y)
119         return self
120
121     def predict(self, X):
122         return np.full(X.shape[0], self.mean_val)
123
124
125 if __name__ == "__main__":
126     np.random.seed(42)
127
128     # Synthetic Dataset Verification
129     N_samples, d_features = 300, 5
130     X_data = np.random.randn(N_samples, d_features)
131     y_data = (X_data[:, 0] + X_data[:, 1] > 0).astype(int)
132

```



```

133 # Train Pure Bagging Engine
134 bagging_engine = UniversalBaggingEngine(
135     base_estimator_class=DummyUnstableTree,
136     estimator_params={'max_depth': 5},
137     n_estimators=40,
138     mode='classification'
139 )
140 bagging_engine.fit(X_data, y_data)
141
142 predictions = bagging_engine.predict(X_data)
143 print("=== PURE BAGGING ENGINE INITIALIZATION SUCCESSFUL ===")
144 print(f"Ensemble Base Estimators : {len(bagging_engine.estimators_)}")
145 print(f"OOB Score Calculated      : {bagging_engine.oob_score_:.4f}")

```

Listing 1: Production-grade NumPy implementation of Pure Bootstrap Aggregating (Bagging).